

Protocolo de Comunicación BLE

Aplicación – Robot Atta-Bot STEM

AttaBot STEM

27 de julio de 2026

Este documento describe algunos conceptos del protocolo de comunicación utilizado entre la aplicación (App) y el robot Atta-Bot STEM. La primera etapa abarca el enlace mediante Bluetooth Low Energy (BLE) entre la aplicación (Central/Client) y el microcontrolador ESP32 del robot (Periférico/Server), implementado mediante la biblioteca nativa `BLEDevice` del core `esp32:esp32`. La segunda etapa describe el formato y significado de los comandos que viajan sobre ese enlace, definidos en el protocolo de instrucciones propio del proyecto.

El objetivo es ofrecer una referencia para que al revisar el código fuente (`CodigoESP32.ino`) se puede reconocer con facilidad qué representa cada línea relacionada con la comunicación. Las fuentes completas utilizadas como base para la construcción del protocolo se encuentran en el apartado de referencias [3.2].

1. Conceptos básicos de BLE

La siguiente tabla resume conceptos utilizados en este proyecto:

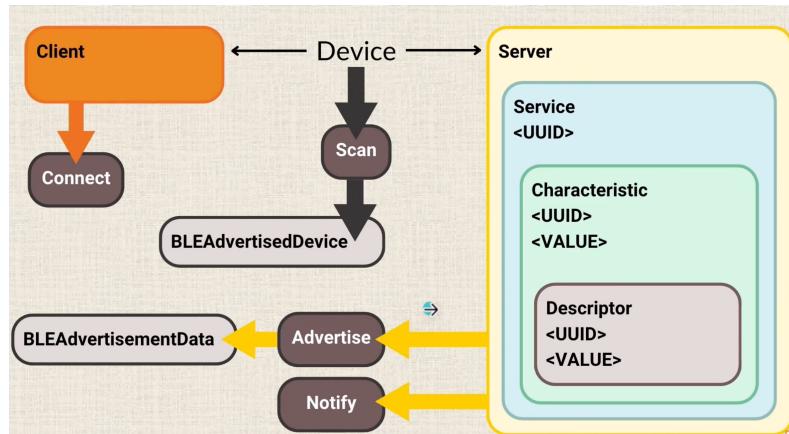


Figura 1: Diagrama de Flujo de Comunicación BLE. Tomado de [3].

Concepto BLE	Dónde aparece en el código y qué significa
Inicialización del stack	Enciende el controlador Bluetooth del ESP32 y define el nombre con el que el dispositivo será visible al escanear. En el código: <code>BLEDevice::init()</code> .
Rol de Periférico/Server	El ESP32 actúa como el dispositivo que espera conexiones, en lugar de buscarlas activamente. En el código: <code>BLEServer</code> , <code>BLEDevice::createServer()</code> .
Callback de conexión	Función que la pila BLE ejecuta automáticamente al conectarse o desconectarse un dispositivo; nunca se llama manualmente. En el código: clase <code>BLEServerCallbacks</code> , métodos <code>onConnect()</code> / <code>onDisconnect()</code> .
Servicio	Agrupar una o más características bajo un identificador único (UUID); es una categoría dentro de los datos que expone el servidor. En el código: <code>BLEService</code> , <code>createService()</code> .
Característico	Una pieza de datos específica dentro de un servicio, con su propio UUID, que puede leerse y/o escribirse – el equivalente a una nota puntual dentro del “tablero” de datos del servidor. En el código: <code>BLECharacteristic</code> , <code>createCharacteristic()</code> .
Propiedades del característico	Determinan qué operaciones puede realizar el dispositivo remoto sobre ese dato: solo lectura, solo escritura, o ambas. En el código: <code>PROPERTY_READ</code> , <code>PROPERTY_WRITE</code> .
Callback de escritura	Se ejecuta automáticamente cuando el dispositivo remoto escribe un nuevo valor en el característico. En el código: clase <code>BLECharacteristicCallbacks</code> , método <code>onWrite()</code> .
Advertising (anuncio)	Proceso mediante el cual el ESP32 transmite activamente su existencia y disponibilidad para conectarse. Sin este paso, el dispositivo es invisible para cualquier escaneo. En el código: <code>getAdvertising()->start()</code> .

Cuadro 1: Correspondencia entre conceptos de BLE y elementos del código fuente.

2. Arquitectura de implementación en el proyecto

El robot utiliza un único servicio con un único característico de lectura/escritura, que transporta texto libre. El significado de cada mensaje no lo define BLE, sino el protocolo de aplicación propio del proyecto (Sección 3). BLE actúa únicamente como canal de transporte, mientras que la interpretación del contenido ocurre en el firmware, dentro de la función `Interpreta_mensajeBLE()`. Cabe destacar que las librerías utilizadas (`<BLEDevice.h>`, `<BLEServer.h>`, `<BLEUtils.h>`) al ser nativas del core `esp32:esp32` no requieren instalación previa.

El flujo general de la comunicación se puede sintetizar de la siguiente forma: Un dispositivo se conecta a un robot Atta. Los avisos de conexión exitosa y recepción de un nuevo comando son detectados con la función `leerBluetooth()`, la cual revisa variables anidadas a los *callback* definidas por la librería constantemente. Seguidamente la App envía un comando, esto es escribir en el característico con UUID único; ejemplo: `ATINIAV300GI090ATFIN` - avanzar 300 cm, girar a la izquierda 90°. Cuando el mensaje es detectado se llama la función `Interpreta_mensajeBLE()` que descompone el mensaje con una concatenación de condicionales, para así determinar finalmente los siguientes pasos de la máquina de estados general.

2.1. Inicialización del servidor BLE

El siguiente fragmento, tomado de `setup()`, muestra la creación del servidor, el servicio y el característico:

```
BLEDevice::init("AttaBotSTEM");

pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());

BLEService *pService = pServer->createService(
    "4fafc201-1fb5-459e-8fcc-c5c9c331914b");

pCharacteristic = pService->createCharacteristic(
    "beb5483e-36e1-4688-b7f5-ea07361b26a8",
    BLECharacteristic::PROPERTY_READ |
    BLECharacteristic::PROPERTY_WRITE
);
pCharacteristic->setCallbacks(new MyCharacteristicCallbacks());

pService->start();

pServer->getAdvertising()->addServiceUUID(
    "4fafc201-1fb5-459e-8fcc-c5c9c331914b");
pServer->getAdvertising()->start();
```

Fragmento de código 1: Inicialización BLE en `setup()`

2.2. Callbacks

Dado que `BLEDevice` funciona mediante eventos asíncronos no es posible “preguntar” en cada iteración de `loop()` si llegó un mensaje nuevo, el proyecto define dos clases de callback. La de escritura sobre el característico es la más relevante para el protocolo de mensajes:

```
class MyCharacteristicCallbacks: public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String valorRecibido = pCharacteristic->getValue();
        if (valorRecibido.length() > 0) {
            mensajeBLEBuffer = string(valorRecibido.c_str());
            nuevoMensajeBLE = true;
        }
    }
};
```

Fragmento de código 2: Callback de escritura sobre el característico

La bandera `nuevoMensajeBLE` se revisa dentro de `loop()` (a través de `leerBluetooth()`) para procesar el mensaje recibido sin depender de un *polling* constante. La documentación oficial de BLE para ESP32 contiene una explicación y justificación detallada del uso de estas clases [2].

3. Protocolo de mensajes de la aplicación

3.1. Consideraciones generales

- Cada comando enviado tiene una longitud fija de 5 caracteres.
- No existen divisores entre comandos (sin comas, espacios, etc.).
- Todo mensaje completo inicia con un comando de encabezado y finaliza con un comando de finalización.

3.2. Lista de comandos

Comando	Detalles
ATINI	Encabezado del mensaje con instrucciones
ATFIN	Finalización del mensaje con instrucciones
OBINI	Inicio de la detección de obstáculos
OBFIN	Finalización de la detección de obstáculos
CI###	Inicio del ciclo. ### indica la cantidad de repeticiones
CIFIN	Finalización del ciclo
WH###	Inicio del ciclo condicional <i>while</i> . ### indica la condición (Nota 3)
WHFIN	Finalización del ciclo condicional <i>while</i>
IF###	Inicio de bifurcación <i>if</i> . ### indica la condición (Nota 3)
EL###	Inicio de bifurcación <i>else if</i> . ### indica la condición (Nota 3)
IFFIN	Finalización de bifurcación <i>if</i>
AV###	Avanzar una distancia de ### cm
RE###	Retroceder una distancia de ### cm
GI###	Giro a la izquierda un ángulo de ### grados
GD###	Giro a la derecha un ángulo de ### grados
HEINI	Inicio del uso del marcador
HEFIN	Finalización del uso del marcador
HE###	Activación del servomotor del set de herramienta. ### indica la acción (Nota 4)
ATCOI	Encabezado del mensaje de control
ATCOF	Finalización del mensaje de control
PARAR	Detener el robot
EJECU	Ejecutar comandos guardados en la memoria del robot

Cuadro 2: Lista de comandos del protocolo de instrucciones.

Nota 1. En todos los comandos con ###, este representa un número entero de 3 dígitos sin signo.

Nota 2. Los comandos relacionados con instrucciones que el robot debe ejecutar en secuencia se envían dentro de los encabezados de “mensaje con instrucciones” (ATINI / ATFIN). Los comandos que deben ejecutarse con prioridad sobre cualquier otra tarea se envían dentro de los encabezados de control (ATCOI / ATCOF).

Nota 3. Para los comandos IF/EL y WH, ### representa las condiciones de los sensores infrarrojos de seguimiento (*tracker*):

Valor ###	Sensor izquierdo	tracker	Sensor derecho	tracker
000	No detecta		No detecta	
001	No detecta		Detecta	
010	Detecta		No detecta	
011	Detecta		Detecta	

Cuadro 3: Condiciones de sensores para bifurcaciones IF / EL.

Valor ###	Tipo de condición	Sensor izquierdo	tracker	Sensor derecho	tracker
100	Hasta / <i>WhileNot</i>	No detecta		No detecta	
000	Mientras / <i>While</i>	No detecta		No detecta	

Cuadro 4: Condiciones de sensores para el ciclo WH.

Nota 4. Para el comando HE###, ### representa una acción:

Valor ###	Acción
101	Abrir garra
001	Cerrar garra
102	Subir grúa
002	Bajar grúa

Cuadro 5: Acciones para el comando HE###.

Referencias

- [1] Arduino, “ArduinoBLE Library Reference,” Arduino Docs. [En línea]. Disponible en: <https://docs.arduino.cc/libraries/arduinoable/>. [Accedido: 25-jul-2026].
- [2] N. Kolban, “BLE C++ Guide,” Repositorio esp32-snippets. [En línea]. Disponible en: <https://github.com/nkolban/esp32-snippets/blob/master/Documentation/BLE%20C++%20Guide.pdf>. [Accedido: 25-jul-2026].
- [3] Programming Electronics Academy, “ESP32 BLE Server + Client,” YouTube, 21-abr-2025. [Video en línea]. Disponible en: <https://www.youtube.com/watch?v=lj99N6pW-Ew&t=742s>. [Accedido: 25-jul-2026].